

Report - BE-SHS

Project BE — Phase 1

Maximilien Larroche 22301310

1. Introduction

My project is to compare several data's structure of int in rust with the same trait. The first part is alone. I have to use the set operation : union, intersection, difference, symmetric difference, also serialization and deserialisation and fragmentation. The second half we have to compare the trait and result by group and decide one trait for everyone to continue.

2. My research

2.1. First Implementation

First of all, I don't know to code in rust, it was the most of my time. To learn I choose to watch video on Youtube and do the rust book.

After that I start to build a trait call StructureDonnee with the basic function like new, add, remove, delete, there_is and also the prototype of fragmenter. I start to implement the binarytree structure because it was the structure I know the most, it was difficult cause to the syntax of rust with the ownership system, mutable type and the management of references.

In the process I discover that rust have a function drop that free the memory so the function delete wasn't necessary. I also add the prototype of map and an iterator because I need to print properly the structure and do some modification, the iterator was needed for the set operation that I implement later.

```
pub trait StructureDonnee {
    fn new() -> Self;
    fn add(&mut self, value: i32);
    fn remove(&mut self, value: i32);
    fn there_is(&self, value: i32) -> bool;
    fn map(&self, f: impl Fn(i32) -> i32 + Copy) -> Self;
    fn iter(&self) -> Box<dyn Iterator<Item = &i32> + '_>;
    fn fragmenter(self, taille_max: usize) -> Vec<Self>
    where
        Self: Sized + Clone;
}
```

I implemented the set operations (union, intersection, difference, symmetric difference) as default methods in the trait, using only add, remove, there_is and iter. This way, any structure implementing the trait automatically inherits these operations without duplicating code. This follows the DRY principle and makes adding new data structures easier.

2.2. Result

I need to verify that my code is correct, to do it I just use a print command and see if the result was good. But I need to do it for every implementation and it's not a good way to do it. So I need to create a real test with comparison that can take any structure that implement the trait to verify the proper functioning but with a lack of time I will do it for the second phase.

I have to test my code for the performance, in Rust there is a library call criterion that measure the time of a function. I decide to implement that in a generic way to not duplicate code for every structure that

I test. The test is about 10 000 element and an sample of 50 for the average. I implement HashSet and TreeSet who is already in the standard library, so it's really quick, to compare with my structure.

The test was run on windows 11 with only Vs Code open, we need to create the same environment for every test to compare it.

Opération	BST	HashSet	BTreeSet
add 10 000	1.05 ms	287 µs	284 µs
remove 10 000	1.35 ms	413 µs	413 µs
there_is	9.0 ns	7.3 ns	7.3 ns
Union	1.14 ms	145 µs	143 µs
Difference	1.35 ms	269 µs	270 µs
symmetric Diff.	33.4 ns	39.0 ns	39.6 ns
Intersection	1.14 ms	145 µs	147 µs
Fragmenter	2.30 ms	109 µs	109 µs

Table 1: Result of benchmarks — 10 000 elements

With the test we see that my structure is the worst because it's not balance, otherwise the HashSet et TreeSet are balanced so the performance are better. My tree have a time complexity of $O(n)$ whereas TreeSet maintains a balanced structure with $O(\log n)$ complexity. The HashSet and TreeSet have very similar performance.

The serialization and deserialization were not implemented due to time constraints.

3. Conclusion

To summarize the difficult part was to learn Rust from scratch and also understand the subject because my tutor don't respond to our mail, we don't have meeting, he had to give us a github link to start the project but we never saw it. To fully finish phase one, I still need to complete the test, implement additionnal data structures and add serialization/deserialization. For the second phase, the goal is to work in several groups and agree on a common trait.